



UNIVERSITY OF PETROLEUM AND ENERGY
STUDIES
DEHRADUN

Applied DevOps

Experiment 3

MTECH - School of Computer Science
CYBER SECURITY AND FORENSICS

Submitted To: Dr. KrishnaRaj

Name: Jigesh Sheoran

SAP ID: 590025428

Objective — To implement a Continuous Integration (CI) pipeline using GitHub Actions for a Python-based scientific calculator, ensuring automated testing and validation of code changes.

Problem Statement:

- Develop a scientific calculator in Python with basic and advanced operations, implement unit and integration testing, and configure a CI pipeline that automatically runs tests whenever code is pushed to GitHub.

Task to be Performed

Step 1: Create project structure

```
Last login: Tue Mar 31 15:11:30 on ttys002
[sheoraninfosec@Jigeshs-MacBook-Air ~ % pwd
/Users/sheoraninfosec
[sheoraninfosec@Jigeshs-MacBook-Air ~ % mkdir ci-calculator
[sheoraninfosec@Jigeshs-MacBook-Air ~ % cd ci-calculator
[sheoraninfosec@Jigeshs-MacBook-Air ci-calculator % ls
[sheoraninfosec@Jigeshs-MacBook-Air ci-calculator % touch basic_operations.py
[sheoraninfosec@Jigeshs-MacBook-Air ci-calculator % touch advanced_operations.py
[sheoraninfosec@Jigeshs-MacBook-Air ci-calculator % touch main.py
[sheoraninfosec@Jigeshs-MacBook-Air ci-calculator % nano basic_operations.py
```

basic_operations.py

```
ci-calculator — nano basic_operations.py — 118x36
UW PICO 5.09 File: basic_operations.py

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        raise ValueError("Division by zero")
    return a / b
```

advanced_operations.py

```
ci-calculator — nano advanced_operations.py — 118x36
UW PICO 5.09 File: advanced_operations.py

import math

def square_root(x):
    return math.sqrt(x)

def power(a, b):
    return math.pow(a, b)

def sine(x):
    return math.sin(x)

def cosine(x):
    return math.cos(x)
```

main.py

```
ci-calculator — nano main.py — 118x36
UW PICO 5.09 File: main.py

# Author : Jigesh Sheoran
from basic_operations import add
from advanced_operations import sine

print(add(5, 3))
print(sine(0))
```

Task 2: Testing

Step 1: Create test folder

```
[sheoraninfosec@Jigeshs-MacBook-Air ci-calculator % mkdir tests
[sheoraninfosec@Jigeshs-MacBook-Air ci-calculator % touch tests/test_basic.py
[sheoraninfosec@Jigeshs-MacBook-Air ci-calculator % touch tests/test_advanced.py
[sheoraninfosec@Jigeshs-MacBook-Air ci-calculator %
```

test_basic.py

```
ci-calculator — nano test_basic.py — 118x36
UW PICO 5.09 File: test_basic.py

import unittest
from basic_operations import *

class TestBasic(unittest.TestCase):

    def test_add(self):
        self.assertEqual(add(2, 3), 5)

    def test_divide(self):
        self.assertEqual(divide(6, 2), 3)

if __name__ == '__main__':
    unittest.main()
```

test_advanced.py

```
ci-calculator — nano test_advanced.py — 118x36
UW PICO 5.09 File: test_advanced.py

import unittest
from advanced_operations import *

class TestAdvanced(unittest.TestCase):

    def test_sqrt(self):
        self.assertEqual(square_root(16), 4)

    def test_power(self):
        self.assertEqual(power(2, 3), 8)

if __name__ == '__main__':
    unittest.main()
```

Integration Tests

```
ci-calculator — nano test_combined.py — 118x36
UW PICO 5.09 File: test_combined.py

import unittest
from basic_operations import add, multiply
from advanced_operations import sine, power

class TestCombinedOperations(unittest.TestCase):

    def test_add_and_sine(self):
        # sin(0 + 0) = 0
        result = sine(add(0, 0))
        self.assertAlmostEqual(result, 0)

    def test_power_and_add(self):
        # (2^3) + 4 = 12
        result = add(power(2, 3), 4)
        self.assertEqual(result, 12)

    def test_mixed_operations(self):
        # sin(2 * 0) = sin(0) = 0
        result = sine(multiply(2, 0))
        self.assertAlmostEqual(result, 0)

    def test_complex_expression(self):
        # (2^2 + 3) = 7 → sin(7)
        result = sine(add(power(2, 2), 3))
        self.assertAlmostEqual(result, sine(7))

if __name__ == '__main__':
    unittest.main()
```

Task 3: Setup CI Pipeline (Github Actions)

Step 1: Create workflow

```
[sheoraninfosec@Jigeshs-MacBook-Air ci-calculator % mkdir -p .github/workflows
[sheoraninfosec@Jigeshs-MacBook-Air ci-calculator % touch .github/workflows/ci.yml
[sheoraninfosec@Jigeshs-MacBook-Air ci-calculator % ]
```

Ci.yml

```
UW PICO 5.09 File: ci.yml

name: CI Pipeline

on:
  push:
    branches: [ master ]
  pull_request:
    branches: [ master ]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.10'

      - name: Install dependencies
        run: pip install pytest

      - name: Run tests
        run: pytest
```

Task 4: Push to github

Step 1 : Create github repo

Quick setup — if you've done this kind of thing before

 Set up in Desktop

 or

HTTPS

SSH

https://github.com/sheoraninfosec/ci-calculator.g



Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# ci-calculator" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/sheoraninfosec/ci-calculator.git
git push -u origin main
```

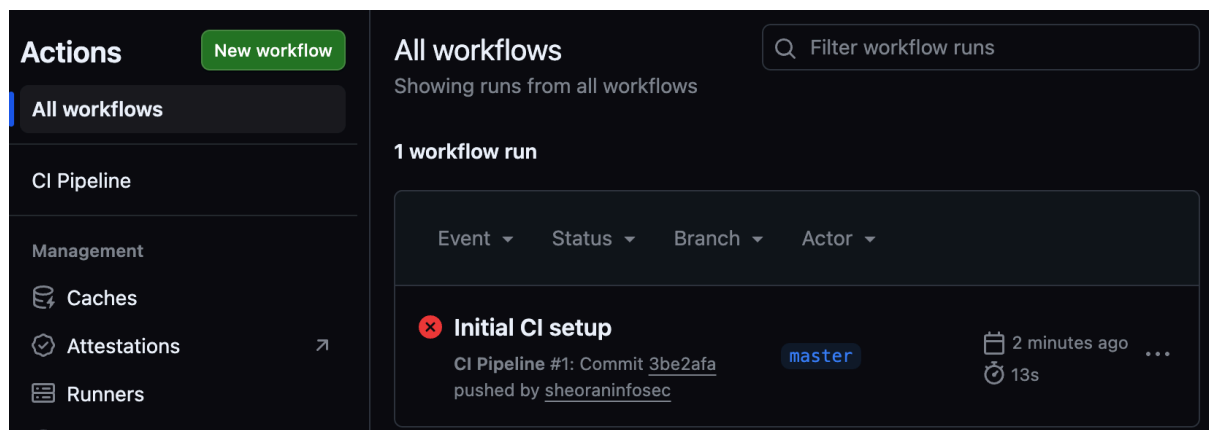


Step 2 : Push to github

```
ci-calculator — -zsh — 118x43

[sheoraninfosec@Jigeshs-MacBook-Air ci-calculator % git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: will change to 'main' in Git 3.0. To configure the initial branch name
hint: to use in all of your new repositories, which will suppress this warning,
hint: call:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /Users/sheoraninfosec/ci-calculator/.git/
[sheoraninfosec@Jigeshs-MacBook-Air ci-calculator % git add .
[sheoraninfosec@Jigeshs-MacBook-Air ci-calculator % git commit -m "Initial CI setup"
[master (root-commit) 3be2afa] Initial CI setup
 9 files changed, 112 insertions(+)
 create mode 100644 .DS_Store
 create mode 100644 .github/workflows/ci.yml
 create mode 100644 advanced_operations.py
 create mode 100644 basic_operations.py
 create mode 100644 main.py
 create mode 100644 tests/.DS_Store
 create mode 100644 tests/test_advanced.py
 create mode 100644 tests/test_basic.py
 create mode 100644 tests/test_combined.py
[sheoraninfosec@Jigeshs-MacBook-Air ci-calculator % git branch -M master
[sheoraninfosec@Jigeshs-MacBook-Air ci-calculator % git remote add origin git@github.com:sheoraninfosec/ci-calculator.g
it
[sheoraninfosec@Jigeshs-MacBook-Air ci-calculator % git push -u origin master
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 8 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (14/14), 2.63 KiB | 1.31 MiB/s, done.
Total 14 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To github.com:sheoraninfosec/ci-calculator.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
sheoraninfosec@Jigeshs-MacBook-Air ci-calculator %
```

Step 3 : Observe how the pipeline runs automatically.



Step 4 : Record what happens when tests pass vs. when tests fail.

Observation : Failed TEST Pipeline

1. The CI pipeline was triggered automatically after pushing code to GitHub
2. The workflow named “Initial CI setup” started execution successfully
3. The pipeline execution status shows a red cross (X Failed)
4. This indicates that one or more test cases did not pass
5. The failure occurred during the automated testing stage of the pipeline
6. The system stopped further execution after detecting the error
7. The failure is recorded in the Actions tab with logs available for debugging

Root Cause

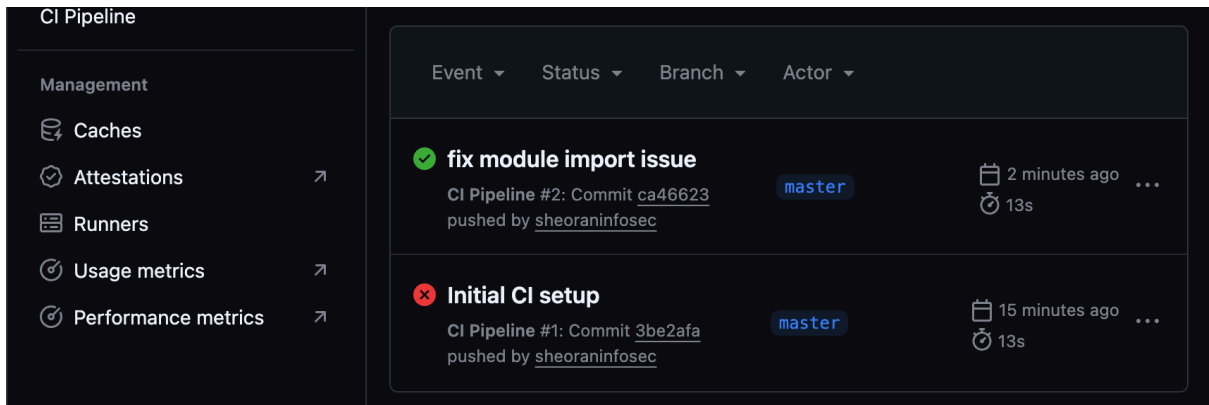
ModuleNotFoundError: No module named 'basic_operations'

Meaning: GitHub Actions cannot find your project files when running tests

In Github Actions - Tests run from **/tests** folder therefore Python cannot see parent directory

```
sheoraninfosec@Jigeshs-MacBook-Air ~ % cd ci-calculator
sheoraninfosec@Jigeshs-MacBook-Air ci-calculator % ls
advanced_operations.py  basic_operations.py  main.py  tests
sheoraninfosec@Jigeshs-MacBook-Air ci-calculator % git add .
sheoraninfosec@Jigeshs-MacBook-Air ci-calculator % git commit -m "fix module import issue"
[master ca46623] fix module import issue
3 files changed, 15 insertions(+)
sheoraninfosec@Jigeshs-MacBook-Air ci-calculator % git push origin master
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 8 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 738 bytes | 738.00 KiB/s, done.
Total 6 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
To github.com:sheoraninfosec/ci-calculator.git
3be2afa..ca46623  master -> master
sheoraninfosec@Jigeshs-MacBook-Air ci-calculator %
```

Test Case Passed :



Observation: Passed TEST Pipeline

- 1. The CI pipeline was automatically triggered after pushing code to GitHub
- 2. The workflow named “fix module import issue” executed successfully
- 3. All stages of the pipeline were completed without errors, including:
- 4. Setting up the environment
- 5. Installing dependencies
- 6. Running test cases
- 7. The “Run tests” step executed successfully, indicating that all unit and integration tests passed
- 8. The pipeline status is shown as Success
- 9. The total execution time was approximately a few seconds, indicating efficient automation
- 10.The previously encountered module import issue was successfully resolved
- 11.The CI system validated the correctness of the code after applying fixes

-----END OF DOCUMENT -----